EN3160 Assignment 2 on Fitting and Alignment

A.A.D.K. Amarasinghe

230033J

01. Blob detection using Laplacian of Gaussians and scale-space

In [1]:
```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
from scipy.ndimage import gaussian_laplace, maximum_filter
from scipy.optimize import minimize
import re
import sys
import glob
import os
```

In [2]:
```
IMG_PATH  = 'a2images/the_berry_farms_sunflower_field.jpeg'
SIGMA_MIN = 2
SIGMA_MAX = 10
N_SCALES  = 10
THRESH    = 0.1

img  = cv.imread(IMG_PATH, cv.IMREAD_REDUCED_COLOR_4)
gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY).astype(np.float64) / 255.0

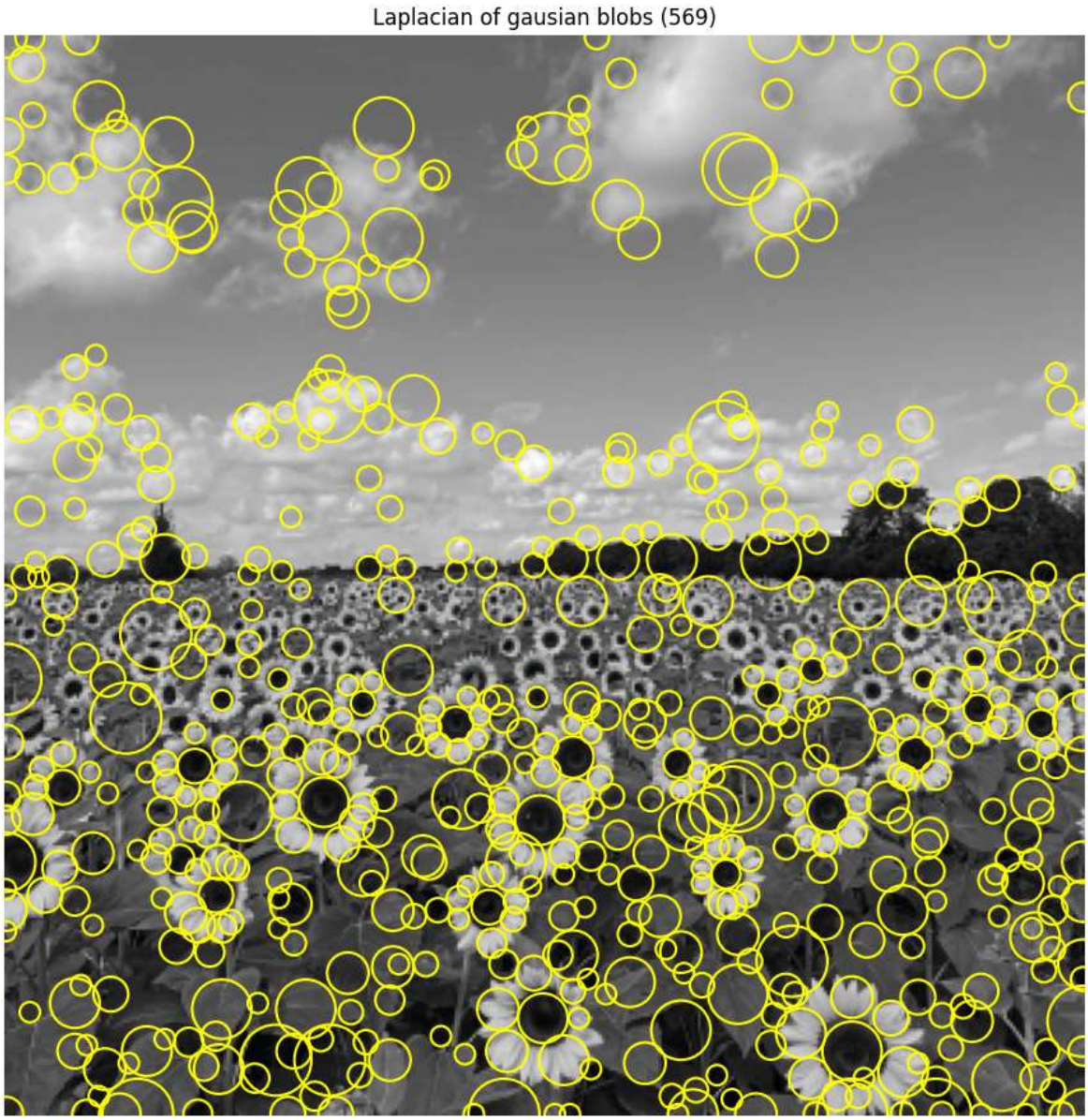
sigmas = np.geomspace(SIGMA_MIN, SIGMA_MAX, N_SCALES)
scale_space = np.stack([(s ** 2) * gaussian_laplace(gray, s) for s in sigmas])

resp = np.abs(scale_space)
is_max = resp == maximum_filter(resp, size=(3, 3, 3))
is_max &= resp > THRESH * resp.max()
is_max[0] = is_max[-1] = False

k, y, x = np.nonzero(is_max)
print('blobs found:', len(k))
```

```
blobs found: 569
```

In [3]:
```
plt.figure(figsize=(9, 9))
plt.imshow(gray, cmap='gray')
for ki, yi, xi in zip(k, y, x):
    plt.gca().add_patch(plt.Circle((xi, yi), np.sqrt(2) * sigmas[ki],
                                   color='yellow', fill=False, lw=1.5))
plt.title(f'Laplacian of gausian blobs ({len(k)})')
plt.axis('off')
plt.tight_layout()
plt.show()
```



Explanation:

The Laplacian of Gaussian algorithm successfully detected 569 scale space blob regions represented by the sudden intensities changes circles across the image. While it effectively captures circular features including the sunflower centers and cloud patterns. But its performance can be significantly enhanced by utilizing the yellow color channel. Focusing on yellow hues reduces cloud clutter and better isolates the distinct sunflower heads.

02. Fit a line and subsequently a circle to a set of noisy points

In [4]:
```
np.random.seed(0)
N = 100
half = N // 2

r_true = 10
cx_true, cy_true = 2, 3
```

```
t = np.random.uniform(0, 2 * np.pi, half)
noise = (r_true / 16) * np.random.randn(half)
X_circ = np.column_stack((cx_true + (r_true + noise) * np.cos(t),
                          cy_true + (r_true + noise) * np.sin(t)))

m_true, b_true = -1, 2
xs = np.linspace(-12, 12, half)
ys = m_true * xs + b_true + np.random.randn(half)
X_line = np.column_stack((xs, ys))

X = np.vstack((X_circ, X_line))
```

In [5]:

```
MAX_R = 50
LINE_THRESH, LINE_MIN = 1.5, 30
CIRC_THRESH, CIRC_MIN = 1.2, 25


def line_from_points(p1, p2):
    dx, dy = p2 - p1
    length = np.hypot(dx, dy)
    if length < 1e-12:
        return None
    a, b = -dy / length, dx / length
    return np.array([a, b, a * p1[0] + b * p1[1]])


def line_error(model, pts):
    a, b, d = model
    return np.abs(a * pts[:, 0] + b * pts[:, 1] - d)


def refine_line(pts):
    centre = pts.mean(axis=0)
    _, _, vt = np.linalg.svd(pts - centre)
    a, b = vt[-1]                       # direction of least variance
    return np.array([a, b, a * centre[0] + b * centre[1]])


def circle_from_points(p1, p2, p3):
    A = 2 * np.array([p2 - p1, p3 - p1])
    rhs = np.array([p2 @ p2 - p1 @ p1, p3 @ p3 - p1 @ p1])
    if abs(np.linalg.det(A)) < 1e-9:
        return None
    cx, cy = np.linalg.solve(A, rhs)
    rad = np.hypot(p1[0] - cx, p1[1] - cy)
    return None if rad > MAX_R else np.array([cx, cy, rad])


def circle_error(model, pts):
    cx, cy, rad = model
    return np.abs(np.hypot(pts[:, 0] - cx, pts[:, 1] - cy) - rad)


def refine_circle(pts, start):
    cost = lambda p: np.sum(circle_error(p, pts) ** 2)
    return minimize(cost, start, method='BFGS').x
```

Part a

In [6]:
```
def ransac(pts, n_sample, fit, error, thresh, min_inliers, n_iter):
    rng = np.random.default_rng()
    best_model, best_mask, best_count = None, None, 0

    for _ in range(n_iter):
        idx = rng.choice(len(pts), n_sample, replace=False)
        model = fit(*pts[idx])
        if model is None:
            continue
        mask = error(model, pts) < thresh
        if mask.sum() > best_count:
            best_model, best_mask, best_count = model, mask, mask.sum()

    if best_count < min_inliers:
        raise RuntimeError('consensus too small')
    return best_model, best_mask
```

Part b

In [7]:
```
def fit_line_ransac(pts, min_inliers=LINE_MIN):
    model, mask = ransac(pts, 2, line_from_points, line_error,
                         LINE_THRESH, min_inliers, 500)
    return refine_line(pts[mask]), mask


def fit_circle_ransac(pts, min_inliers=CIRC_MIN):
    model, mask = ransac(pts, 3, circle_from_points, circle_error,
                         CIRC_THRESH, min_inliers, 1000)
    return refine_circle(pts[mask], model), mask
```

Part C

In [8]:
```
def plot_fit(circ, circ_pts, line, line_pts, title, fname):
    fig, ax = plt.subplots(figsize=(8, 8))
    ax.scatter(X[:, 0], X[:, 1], c='tab:blue', s=25, label='All points')
    ax.scatter(*circ_pts.T, c='c', s=25, label='Circle inliers')
    ax.scatter(*line_pts.T, c='y', s=25, label='Line inliers')

    xl = np.array([-14, 14])
    ax.add_patch(plt.Circle((cx_true, cy_true), r_true, color='g', fill=False,
                            label='Ground truth circle'))
    ax.plot(xl, m_true * xl + b_true, color='m', alpha=0.4, lw=1,
            label='Ground truth line')

    # estimates
    ax.add_patch(plt.Circle(circ[:2], circ[2], color='m', fill=False,
                            label='RANSAC circle'))
    a, b, d = line
    ax.plot(xl, (d - a * xl) / b, color='purple', label='RANSAC line')

    ax.set_aspect('equal')
    ax.set_xlim(-15, 15)
    ax.set_ylim(-14, 17)
    ax.set_title(title)
    ax.legend(fontsize=7, loc='upper right')
    fig.tight_layout()
    fig.savefig(fname, dpi=150)
```

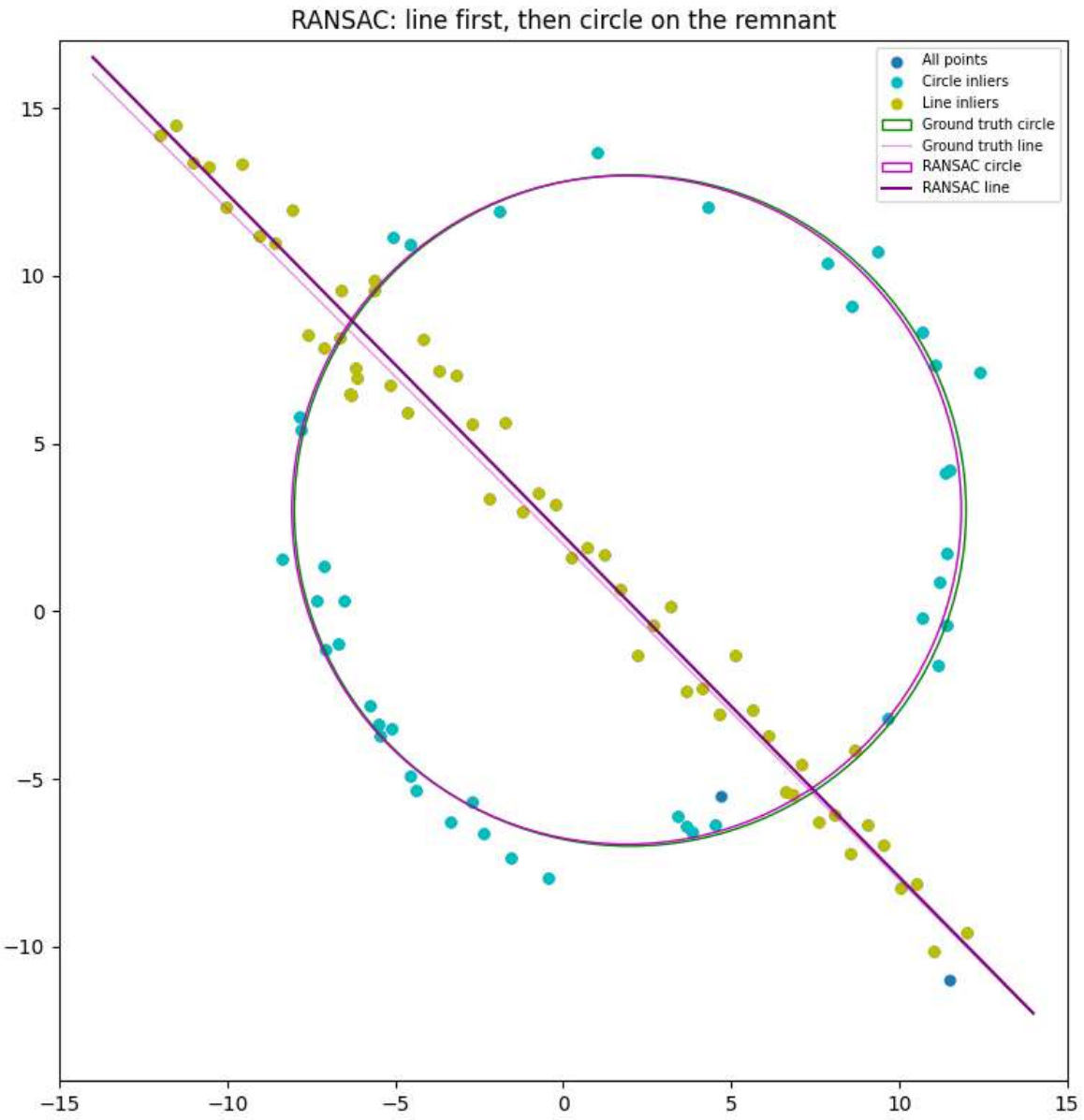
line first, then circle

In [9]:
```
line, line_mask = fit_line_ransac(X)
a, b, d = line
print('(a) RANSAC line: a=%.4f b=%.4f d=%.4f  inliers=%d'
      % (a, b, d, line_mask.sum()))
norm = np.hypot(m_true, 1)
print('    ground truth: normal=[%.4f, %.4f], d=%.4f (sign may be flipped)'
      % (m_true / norm, -1 / norm, -b_true / norm))

remnant = X[~line_mask]
circ, circ_mask = fit_circle_ransac(remnant)
print('(b) RANSAC circle: centre=(%.3f, %.3f) r=%.3f  inliers=%d of %d remnant p
      % (*circ, circ_mask.sum(), len(remnant)))
print('    ground truth : centre=(%d, %d) r=%d' % (cx_true, cy_true, r_true))

plot_fit(circ, remnant[circ_mask], line, X[line_mask],
         'RANSAC: line first, then circle on the remnant', 'q2_line_then_circle.
```

```
(a) RANSAC line: a=0.7133 b=0.7009 d=1.5877  inliers=56
    ground truth: normal=[-0.7071, -0.7071], d=-1.4142 (sign may be flipped)
(b) RANSAC circle: centre=(1.893, 3.018) r=9.964  inliers=42 of 44 remnant points
    ground truth : centre=(2, 3) r=10
```



circle first, then line

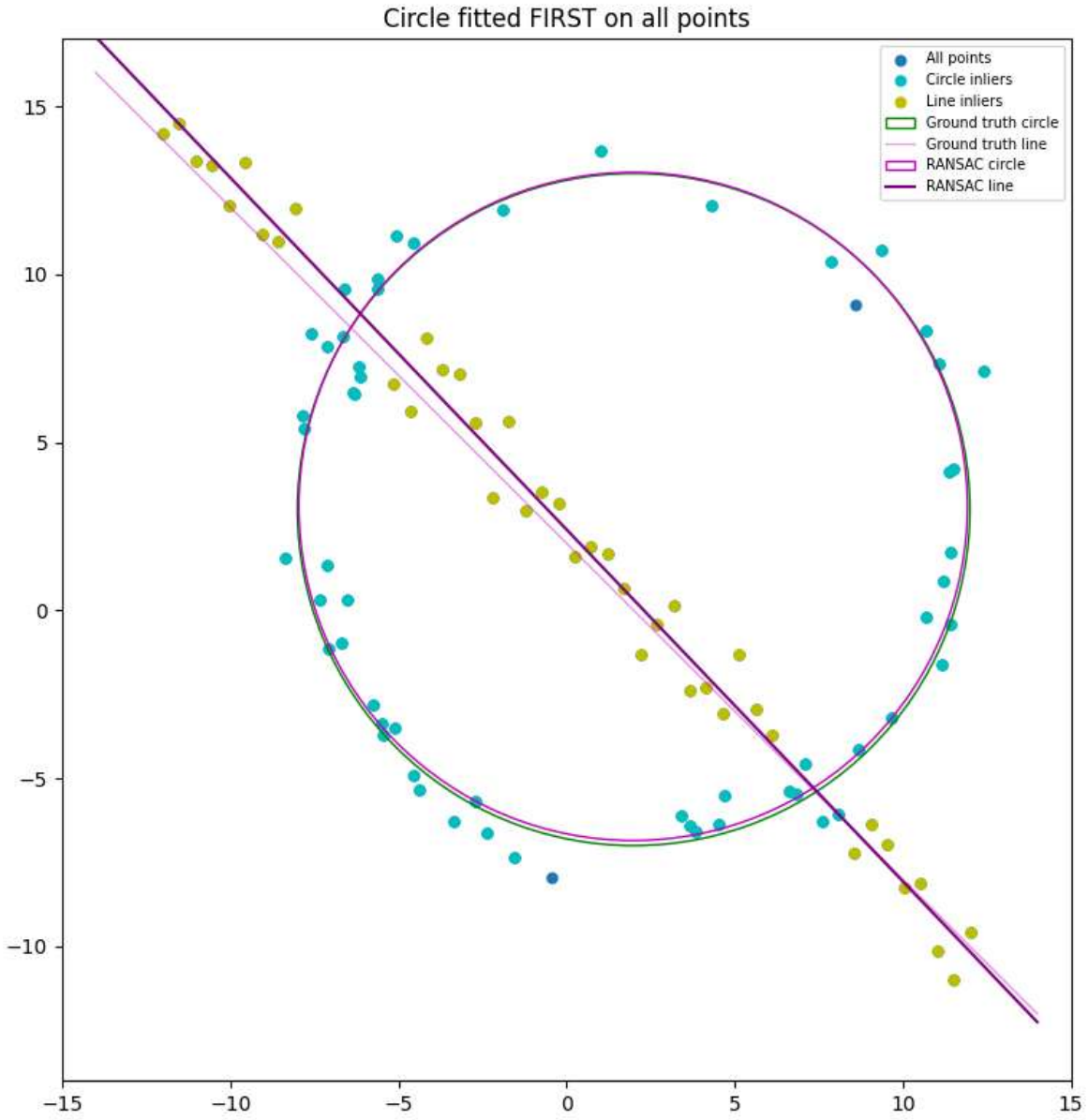
In [10]:
```
print('\n(d) Fitting the circle FIRST on all points:')
circ2, circ_mask2 = fit_circle_ransac(X, min_inliers=20)
print('    circle: centre=(%.3f, %.3f) r=%.3f  inliers=%d'
      % (*circ2, circ_mask2.sum()))

remnant2 = X[~circ_mask2]
line2, line_mask2 = fit_line_ransac(remnant2, min_inliers=15)
print('    line on remnant: inliers=%d of %d remnant points'
      % (line_mask2.sum(), len(remnant2)))

plot_fit(circ2, X[circ_mask2], line2, remnant2[line_mask2],
         'Circle fitted FIRST on all points', 'q2_circle_first.png')

plt.show()
```

```
(d) Fitting the circle FIRST on all points:
    circle: centre=(1.984, 3.093) r=9.942  inliers=57
    line on remnant: inliers=41 of 43 remnant points
```



Comparison

In the first approach Circle fitted first on all points the algorithm attempts to extract the dominant circular structure from the entire dataset first. Because the intersecting linear points act as a heavy concentration of outliers relative to the circle model so they bias the initial RANSAC circle estimation and it causing a slight divergence from the green ground truth circle.

But in the second approach Line first then circle on the remnant extracts the linear structure first and removes its inliers when identifing circle and subsequently fits the circle to the remaining points. This sequential removal prevents the line points from corrupting the circular fit. So it result in a RANSAC circle that aligns almost perfectly with the ground truth circle.

03. Homography

In [11]:
```
PAIRS = [
    ('a2images/wadham.jpg', 'a2images/flag.png', None, 'q3_result_1.png'),
    ('a2images/001.jpg',    'a2images/flag.png', None, 'q3_result_2.png'),
]
ALPHA = 0.7
USE_SAVED_POINTS = False
```

In [12]:
```
def _display_file(path):
    try:
        from IPython import get_ipython
        from IPython.display import display, Image
        if get_ipython() is not None:
            display(Image(filename=path))
            return True
    except Exception:
        pass
    return False


def _click_opencv(img_bgr, title='Click 4 points: TL, TR, BR, BL  (Esc = cancel)
    pts = []
    disp = img_bgr.copy()

    def on_mouse(event, x, y, flags, param):
        if event == cv.EVENT_LBUTTONDOWN and len(pts) < 4:
            pts.append([x, y])
            cv.circle(disp, (x, y), 6, (0, 0, 255), -1)
            cv.putText(disp, str(len(pts)), (x + 10, y - 10),
                       cv.FONT_HERSHEY_SIMPLEX, 0.9, (0, 0, 255), 2)

    cv.namedWindow(title, cv.WINDOW_NORMAL)
    cv.setMouseCallback(title, on_mouse)
    while len(pts) < 4:
        cv.imshow(title, disp)
        if cv.waitKey(20) & 0xFF == 27:
            break
    cv.imshow(title, disp)
    cv.waitKey(400)
    cv.destroyAllWindows()
    cv.waitKey(1)
```

```
    if len(pts) < 4:
        raise RuntimeError('Need 4 points')
    return np.array(pts, dtype=np.float32)


def _click_matplotlib_tk(img_bgr):
    old = plt.get_backend()
    pts = []
    try:
        plt.switch_backend('TkAgg')
        fig, ax = plt.subplots(1, 1, figsize=(10, 8))
        ax.imshow(cv.cvtColor(img_bgr, cv.COLOR_BGR2RGB))
        ax.set_title('Click 4 points: TL, TR, BR, BL')
        pts = plt.ginput(4, timeout=120)          # gives up after 2 minutes
        plt.close(fig)
    finally:
        try:
            plt.switch_backend(old)
        except Exception:
            pass
    return np.array(pts, dtype=np.float32) if len(pts) == 4 else None


def _click_manual(img_bgr):
    h, w = img_bgr.shape[:2]
    fig, ax = plt.subplots(1, 1, figsize=(12, 9))
    ax.imshow(cv.cvtColor(img_bgr, cv.COLOR_BGR2RGB))
    ax.set_xticks(np.arange(0, w, 50)); ax.set_yticks(np.arange(0, h, 50))
    ax.grid(True, color='yellow', alpha=0.5, lw=0.5)
    ax.tick_params(labelsize=6, labeltop=True, labelright=True)
    plt.setp(ax.get_xticklabels(), rotation=90)
    ax.set_title(f'Image {w}x{h}: read pixel coordinates (x right, y down)')
    plt.close(fig)

    if not _display_file('q3_pick_points.png'):
        print('Open q3_pick_points.png to read the pixel coordinates.')

    s = input('Type 4 points as: x1,y1 x2,y2 x3,y3 x4,y4  (TL, TR, BR, BL): ')
    nums = re.findall(r'-?\d+\.?\d*', s)
    if len(nums) != 8:
        raise ValueError('4 points needed')
    return np.array(nums, dtype=np.float32).reshape(4, 2)


def click_four_points(img_bgr):
    try:
        return _click_opencv(img_bgr)
    except cv.error:
        print('OpenCV window not available')
    try:
        pts = _click_matplotlib_tk(img_bgr)
        if pts is not None:
            return pts
    except Exception as e:
        print('matplotlib Tk window not available:', type(e).__name__)
    return _click_manual(img_bgr)
```

In [13]:

```
def homography_dlt(src, dst):
    A = []
    for (x, y), (u, v) in zip(src, dst):
```

```
        A.append([-x, -y, -1, 0, 0, 0, u * x, u * y, u])
        A.append([0, 0, 0, -x, -y, -1, v * x, v * y, v])
    _, _, Vt = np.linalg.svd(np.asarray(A, dtype=np.float64))
    H = Vt[-1].reshape(3, 3)
    return H / H[2, 2]


def superimpose(scene, overlay, dst_pts, alpha=ALPHA):
    h, w = overlay.shape[:2]
    src_pts = np.array([[0, 0], [w - 1, 0], [w - 1, h - 1], [0, h - 1]], dtype=n

    H = homography_dlt(src_pts, dst_pts)
    # sanity check against OpenCV
    H_cv = cv.getPerspectiveTransform(src_pts, dst_pts.astype(np.float32))
    print('H (own DLT):\n', np.round(H, 5))
    print('max |H - H_opencv| =', np.abs(H - H_cv / H_cv[2, 2]).max())

    size = (scene.shape[1], scene.shape[0])
    warped = cv.warpPerspective(overlay, H, size)
    mask = cv.warpPerspective(np.full((h, w), 255, np.uint8), H, size)
    mask = (mask > 0)[..., None]

    blended = scene.copy().astype(np.float32)
    blended = np.where(mask, alpha * warped + (1 - alpha) * blended, blended)
    return blended.astype(np.uint8), warped, H
```

In [14]:

```
def run_pair(scene_path, overlay_path, pts, out_name):
    scene = cv.imread(scene_path)
    overlay = cv.imread(overlay_path)
    if scene is None or overlay is None:
        print(f'cannot read {scene_path} or {overlay_path}')
        return
    if pts is None or not USE_SAVED_POINTS:
        pts = click_four_points(scene)
        print('clicked points:', pts.tolist(), ' PAIRS to reuse')
    pts = np.asarray(pts, dtype=np.float32)

    result, warped, H = superimpose(scene, overlay, pts)

    fig, axs = plt.subplots(1, 3, figsize=(16, 5))
    axs[0].imshow(cv.cvtColor(scene, cv.COLOR_BGR2RGB)); axs[0].set_title('Scene
    axs[0].plot(*np.vstack([pts, pts[:1]]).T, 'r.-')
    axs[1].imshow(cv.cvtColor(overlay, cv.COLOR_BGR2RGB)); axs[1].set_title('Ove
    axs[2].imshow(cv.cvtColor(result, cv.COLOR_BGR2RGB)); axs[2].set_title('Supe
    for a in axs:
        a.axis('off')
    plt.tight_layout()
    plt.savefig(out_name, dpi=150)
    if _display_file(out_name):
        plt.close(fig)
    else:
        plt.show()

if len(sys.argv) == 3:
        run_pair(sys.argv[1], sys.argv[2], None, 'q3_result.png')
else:
    for scene_p, over_p, pts, out in PAIRS:
        print('Pair:', scene_p, '+', over_p)
        run_pair(scene_p, over_p, pts, out)
```

```
Pair: a2images/wadham.jpg + a2images/flag.png
OpenCV window not available
clicked points: [[193.72967529296875, 214.3748321533203], [499.2120056152344, 28
2.02490234375], [494.9838562011719, 455.3782043457031], [184.2163848876953, 438.4
656677246094]]  PAIRS to reuse
H (own DLT):
 [[ 2.3566000e-01 -1.2420000e-02  1.9372968e+02]
 [ 7.8450000e-02  2.2772000e-01  2.1437483e+02]
 [ 1.5000000e-04 -1.0000000e-05  1.0000000e+00]]
max |H - H_opencv| = 6.290434839684167e-09
```




```
Pair: a2images/001.jpg + a2images/flag.png
OpenCV window not available
clicked points: [[522.4667358398438, 156.23806762695312], [524.5807495117188, 18
3.7209014892578], [862.8311157226562, 119.24193572998047], [870.2303466796875, 7
2.73251342773438]]  PAIRS to reuse
H (own DLT):
 [[ 6.5100000e-03  1.6290000e-02  5.2246674e+02]
 [ 1.6210000e-02 -1.1602000e-01  1.5623807e+02]
 [ 1.0000000e-05 -4.0000000e-04  1.0000000e+00]]
max |H - H_opencv| = 9.772804787644418e-09
```



Non-Technical Rationale for Image Pair Selection.

When choosing custom image pairs for this taskt I used following pairs.

1. Clear Planar Surfaces- Choose scene images containing prominent, flat, two dimensional surfaces like brick walls, windows. This guarantees that the homography model which assumes a flat geometric plane accurately maps the overlay without distortion artifacts.
2. Distinct Corner Points- Select scenes where the four target corners are clearly visible and unambiguous so that user clicked points are precise.
3. Visual Contrast- Pick overlay images that contrast well against the target surface background. For instance superimposing a vibrant flag onto a neutral toned architectural stone wall ensures high visibility and highlights the realism of the perspective warp.

4. Has different Angles- Choose a mix of views and highly angled side views. When testing oblique angles effectively demonstrates how this algorithm handles extreme perspective compressions.

04. PCB Alignment

In [15]:
```
IMG_DIR = 'a2images/pcbs'
TEMPLATE_NAME = 'AECC1.jpg'
RATIO = 0.75
RANSAC_THRESH = 5.0
MIN_MATCHES = 10
OUT_DIR = 'q4_results'
```

In [16]:
```
def load_images():
    template = cv.imread(os.path.join(IMG_DIR, TEMPLATE_NAME))
    if template is None:
        raise FileNotFoundError(os.path.join(IMG_DIR, TEMPLATE_NAME))
    paths = []
    for ext in ('*.jpg', '*.jpeg', '*.png', '*.JPG', '*.JPEG', '*.PNG'):
        paths += glob.glob(os.path.join(IMG_DIR, ext))
    paths = sorted(p for p in set(paths) if os.path.basename(p) != TEMPLATE_NAME
    return template, paths
```

Part a -detect SIFT

In [17]:
```
def detect_sift(img_gray):
    sift = cv.SIFT_create()
    return sift.detectAndCompute(img_gray, None)
```

Part b

In [18]:
```
def match_and_find_homography(kp_img, des_img, kp_tpl, des_tpl):
    bf = cv.BFMatcher(cv.NORM_L2)
    knn = bf.knnMatch(des_img, des_tpl, k=2)
    good = [m for m, n in (p for p in knn if len(p) == 2) if m.distance < RATIO
    if len(good) < MIN_MATCHES:
        return None, None, good
    src = np.float32([kp_img[m.queryIdx].pt for m in good]).reshape(-1, 1, 2)
    dst = np.float32([kp_tpl[m.trainIdx].pt for m in good]).reshape(-1, 1, 2)
    H, mask = cv.findHomography(src, dst, cv.RANSAC, RANSAC_THRESH)
    return H, mask.ravel().astype(bool), good


def process(template, img_path):
    img = cv.imread(img_path)
    t_gray = cv.cvtColor(template, cv.COLOR_BGR2GRAY)
    i_gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)

    kp_t, des_t = detect_sift(t_gray)
    kp_i, des_i = detect_sift(i_gray)
    print(f'{os.path.basename(img_path)}: {len(kp_i)} keypoints (template: {len(

    H, inl_mask, good = match_and_find_homography(kp_i, des_i, kp_t, des_t)
    if H is None:
        print('not enough matches skipped')
```

```
        return None
    print(f'good matches: {len(good)}  RANSAC inliers: {int(inl_mask.sum())}')
    print('H =\n', np.round(H, 4))

    vis = cv.drawMatches(img, kp_i, template, kp_t, good, None,
                         matchesMask=inl_mask.astype(int).tolist(),
                         matchColor=(0, 255, 0), singlePointColor=None,
                         flags=cv.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)

    h, w = template.shape[:2]
    aligned = cv.warpPerspective(img, H, (w, h))
    diff = cv.absdiff(aligned, template)
    diff_gray = cv.cvtColor(diff, cv.COLOR_BGR2GRAY)
    return dict(name=os.path.basename(img_path), img=img, matches=vis,
                aligned=aligned, diff=diff, diff_gray=diff_gray, H=H)
```

In [19]:

```
def show(template, res):
    fig = plt.figure(figsize=(16, 9))
    ax = fig.add_subplot(2, 1, 1)
    ax.imshow(cv.cvtColor(res['matches'], cv.COLOR_BGR2RGB)); ax.axis('off')
    ax.set_title(f"SIFT matches (RANSAC inliers): {res['name']}  ->  template")
    for i, (im, title, cmap) in enumerate([
            (cv.cvtColor(template, cv.COLOR_BGR2RGB), 'Template', None),
            (cv.cvtColor(res['aligned'], cv.COLOR_BGR2RGB), 'Aligned image', Non
            (res['diff_gray'], 'Difference |aligned - template|', 'gray')]):
        a = fig.add_subplot(2, 3, 4 + i)
        a.imshow(im, cmap=cmap); a.set_title(title); a.axis('off')
    plt.tight_layout()
    plt.savefig(os.path.join(OUT_DIR, f"{os.path.splitext(res['name'])[0]}_resul
    plt.show()
```

In [20]:

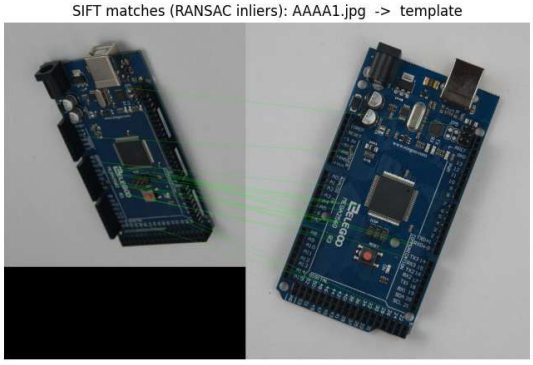
```
os.makedirs(OUT_DIR, exist_ok=True)
template, paths = load_images()
print(f'Template: {TEMPLATE_NAME}, {len(paths)} images to align\n')

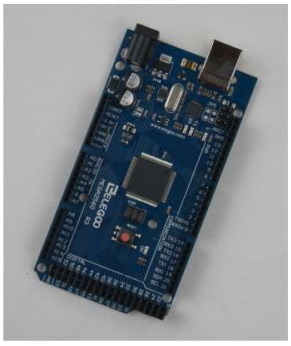
results = []
for p in paths:
    res = process(template, p)
    if res is not None:
        show(template, res)
        results.append(res)

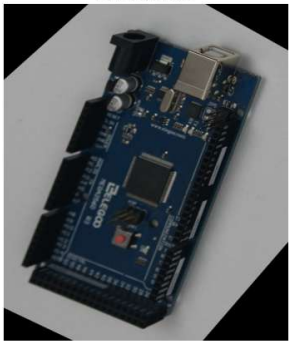
# all difference images in one figure
if results:
    n = len(results)
    cols = min(3, n)
    rows = int(np.ceil(n / cols))
    fig, axs = plt.subplots(rows, cols, figsize=(5 * cols, 4 * rows), squeeze=Fa
    for a in axs.ravel():
        a.axis('off')
    for a, r in zip(axs.ravel(), results):
        a.imshow(r['diff_gray'], cmap='gray'); a.set_title(r['name'])
    plt.suptitle('Difference images (aligned vs template)')
    plt.tight_layout()
    plt.savefig(os.path.join(OUT_DIR, 'all_differences.png'), dpi=130)
    plt.show()
```

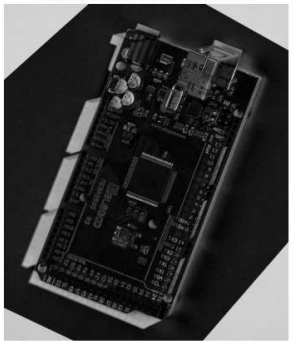
```
Template: AECC1.jpg, 5 images to align

AAAA1.jpg: 2414 keypoints (template: 14272)
good matches: 51  RANSAC inliers: 18
H =
 [[ 1.2005000e+00 -1.1096000e+00  1.0468395e+03]
 [ 6.4290000e-01  9.5130000e-01 -3.1815520e+02]
 [ 0.0000000e+00 -1.0000000e-04  1.0000000e+00]]
```

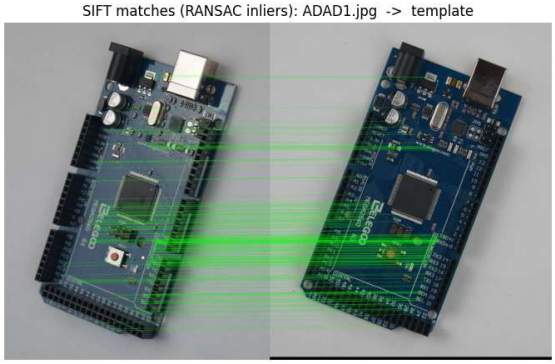






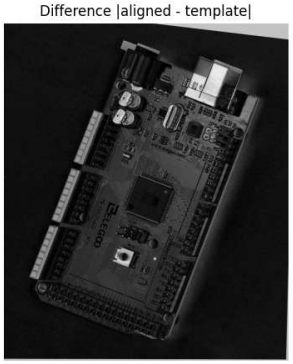



```
ADAD1.jpg: 8486 keypoints (template: 14272)
good matches: 369  RANSAC inliers: 194
H =
 [[ 1.226400e+00 -1.610000e-02 -5.789270e+01]
 [ 1.391000e-01  1.049700e+00 -1.442432e+02]
 [ 1.000000e-04  0.000000e+00  1.000000e+00]]
```

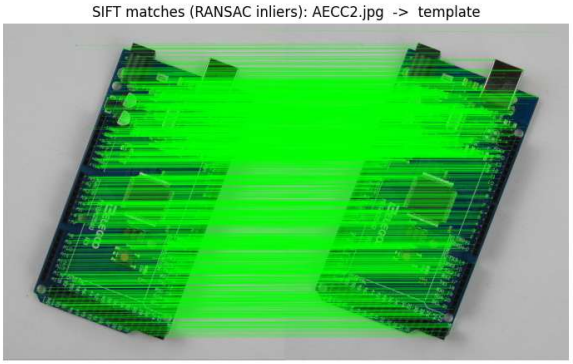








```
AECC2.jpg: 12315 keypoints (template: 14272)
good matches: 4322  RANSAC inliers: 3847
H =
 [[ 1.00060e+00 -1.50000e-02  3.21100e+01]
 [ 1.66000e-02  1.00000e+00 -1.81596e+01]
 [ 0.00000e+00  0.00000e+00  1.00000e+00]]
```

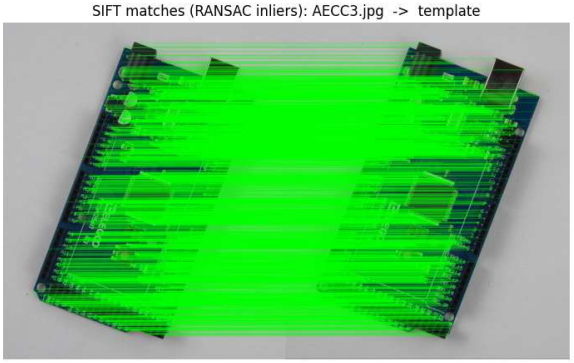




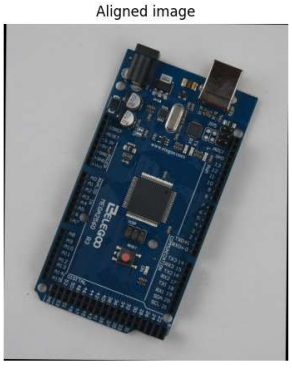





```
AECC3.jpg: 15198 keypoints (template: 14272)
good matches: 5629  RANSAC inliers: 5006
H =
 [[ 9.99600e-01 -1.00000e-02  2.24371e+01]
 [ 1.04000e-02  9.99200e-01 -1.13615e+01]
 [ 0.00000e+00  0.00000e+00  1.00000e+00]]
```

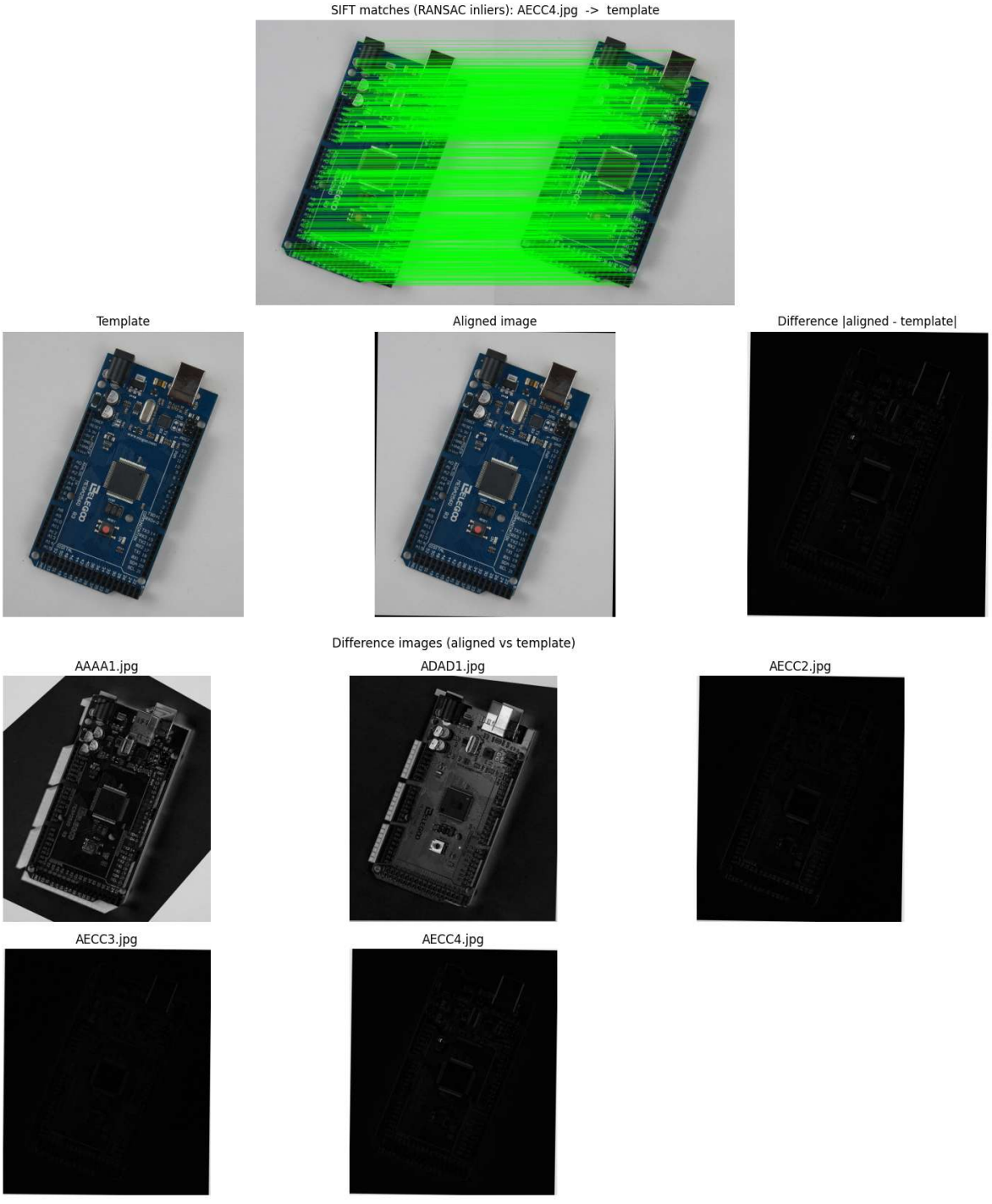








```
AECC4.jpg: 11610 keypoints (template: 14272)
good matches: 3830  RANSAC inliers: 3402
H =
 [[ 9.99400e-01 -1.40000e-02  3.15510e+01]
 [ 1.39000e-02  9.99000e-01 -1.39158e+01]
 [ 0.00000e+00  0.00000e+00  1.00000e+00]]
```



Result Comparison

- Matching Points in SIFT and RANSAC AAAA1.jpg Shows only a few green matching lines. This low number of matches means the image has major differences in size, angle or shape compared to the template. AECC2.jpg, AECC3.jpg and AECC4.jpg Show many green lines covering the board. This high number of good matches proves these images are very similar to the template.

- Image Alignment (Homography) AAAA1.jpg- The program struggles to line up this image with the template because there are not enough matching points, leaving the image tilted with cut off edges. AECC2.jpg, AECC3.jpg and AECC4.jpg- The program easily transforms and stretches these images so they line up almost perfectly with the template's size and angle.
- Finding Differences AAAA1.jpg ADAD1.jpg- Large bright white spots appear in the difference images. These white areas show where the images failed to line up or where parts are different. AECC2.jpg, AECC3.jpg and AECC4.jpg- The difference images are almost completely dark. This solid dark output proves that the aligned images match the template almost perfectly

Part e

High differences around components like the USB Type B port, capacitors and reset button occur because homography assumes a flat 2D plane. Taller 3D elements introduce perspective parallax, height variations and shifting specular reflections that cannot be properly warped resulting in severe local misalignment and residual pixel errors.